

Practical - 12

Aim: There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

PROGRAM CODE :

```
#include <stdio.h>

int uniquePaths(int m, int n) {

    int dp[m][n];

    for (int i = 0; i < m; i++) {
        dp[i][0] = 1;
    }
    for (int j = 0; j < n; j++) {
        dp[0][j] = 1;
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {

            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }

    return dp[m - 1][n - 1];
}
```

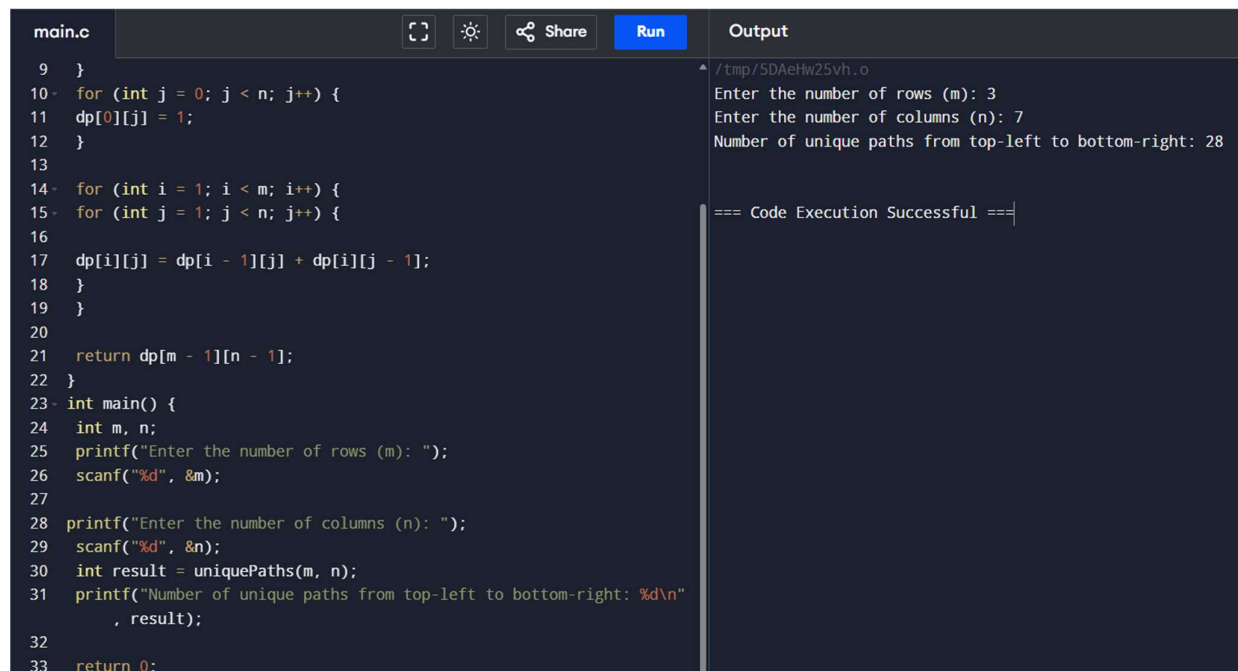
```
}  
  
int main() {  
    int m, n;  
    printf("Enter the number of rows (m): ");  
    scanf("%d", &m);  
  
    printf("Enter the number of columns (n): ");  
    scanf("%d", &n);  
    int result = uniquePaths(m, n);  
    printf("Number of unique paths from top-left to bottom-right: %d\n", result);  
  
    return 0;  
}
```

OUTPUT :-

Enter the number of rows (m): 3

Enter the number of columns (n): 7

Number of unique paths from top-left to bottom-right: 28



The screenshot shows a code editor with a dark theme. On the left, the code for 'main.c' is displayed, featuring a dynamic programming solution for the unique paths problem. The code includes a 2D array 'dp' and nested loops to calculate the number of unique paths from the top-left to the bottom-right of a grid. On the right, the 'Output' panel shows the program's execution with user input (3 rows, 7 columns) and the resulting output (28 unique paths). A status bar at the bottom of the output panel indicates 'Code Execution Successful'.

```
main.c  
9  }  
10 for (int j = 0; j < n; j++) {  
11     dp[0][j] = 1;  
12 }  
13  
14 for (int i = 1; i < m; i++) {  
15     for (int j = 1; j < n; j++) {  
16  
17         dp[i][j] = dp[i - 1][j] + dp[i][j - 1];  
18     }  
19 }  
20  
21 return dp[m - 1][n - 1];  
22 }  
23 int main() {  
24     int m, n;  
25     printf("Enter the number of rows (m): ");  
26     scanf("%d", &m);  
27  
28     printf("Enter the number of columns (n): ");  
29     scanf("%d", &n);  
30     int result = uniquePaths(m, n);  
31     printf("Number of unique paths from top-left to bottom-right: %d\n",  
32         , result);  
33     return 0;  
}
```

Output
/tmp/5DAeHw25vh.o
Enter the number of rows (m): 3
Enter the number of columns (n): 7
Number of unique paths from top-left to bottom-right: 28
=== Code Execution Successful ===